Appendix. Discussion Questions

Chapter 1: Introduction

1. What are the four dimensions that define software architecture?
2. What is the difference between an architecture decision and a design principle?
3. List the eight core expectations of a software architect.
4. What is the First Law of Software Architecture?

Chapter 2: Architectural Thinking

1. Name three criteria for determining whether a decision is more about architecture or more about design.
2. List the three levels of knowledge in the knowledge triangle and provide an example of each.
3. Why is it more important for an architect to focus on technical breadth rather than technical depth?
4. What are some of the ways of maintaining your technical depth and remaining hands-on as an architect?

Chapter 3: Modularity

1. Describe the difference between modularity and granularity, and provide an example of each.
2. What is the difference between coupling and cohesion?
3. What does the term *connascence* mean?
4. What is the difference between static and dynamic connascence?
5. What is the strongest form of connascence?
6. What is the weakest form of connascence?
7. Which is preferable within a code base: static or dynamic connascence?

Chapter 4: Architecture Characteristics Defined

1. What three criteria must an attribute meet to be considered an architecture characteristic?
2. What is the difference between an implicit characteristic and an explicit one? Provide an example of each.
3. Provide an example of an operational characteristic.
4. Provide an example of a structural characteristic.
5. Provide an example of a cross-cutting characteristic.
6. Why is it impossible to create an industry-wide standard list of architecture characteristics?

Chapter 5: Identifying Architectural Characteristics

1. Give a reason why it is a good practice to limit the number of characteristics ("-ilities") an architecture should support.
2. True or false? Most architecture characteristics come from business requirements and user stories.
3. If a business stakeholder states that time to market (that is, getting new features and bug fixes pushed out to users as fast as possible) is the most important business concern, which architecture characteristics would the architecture need to support?
4. What is the difference between scalability and elasticity?
5. What is the definition of a *composite* architectural characteristic? Provide one example.

Chapter 6: Measuring and Governing Architecture Characteristics

1. Why is cyclomatic complexity such an important metric to analyze for architecture?
2. What are architecture fitness functions? How can they be used to analyze an architecture?
3. Provide an example of an architecture fitness function to measure the scalability of an architecture.
4. What is the most important criterion for an architecture characteristic to allow architects and developers to create fitness functions?

Chapter 7: The Scope of Architectural Characteristics

1. What is an architectural quantum, and why is it important to architecture?
2. Assume a system consisting of a single user interface with four independently deployed services, each containing its own separate database. Would this system have a single quantum or four quanta? Why?
3. What is the difference between *static* and *dynamic* coupling? Provide an example of each.
4. Why does *synchronous* communication have a bigger potential impact on operational architectural characteristics than *asynchronous* ones?

Chapter 8: Component-Based Thinking

1. We define the term *component* as a building block of an application—something the application does. A component usually consists of a group of classes or source files. How do components typically manifest within an application or service?
2. What is the difference between technical partitioning and domain partitioning? Provide an example of each.
3. What is the advantage of domain partitioning?
4. Under what circumstances would technical partitioning be a better choice than domain partitioning?
5. What is the Entity Trap? Why is it not a good approach for component identification?
6. When might you choose the Workflow approach over the Actor/Action approach when identifying core components?

Chapter 9: Foundations

1. List the eight fallacies of distributed computing.
2. Name three challenges that distributed architectures have that monolithic architectures don't.
3. What is stamp coupling? What are some ways of addressing stamp coupling?
4. What is the difference between technical versus domain partitioning?
5. List three characteristics that distinguish an architectural style from a pattern.

Chapter 10: Layered Architecture Style

1. What is the difference between an open layer and a closed layer?
2. Describe the concept of *layers of isolation* and its benefits.
3. What is the Architecture Sinkhole antipattern?
4. What are some of the main architecture characteristics that would drive you to use a layered architecture?
5. Why isn't testability well supported in the layered architecture style?
6. Why isn't agility well supported in the layered architecture style?

Chapter 11: Modular Monolith Architecture Style

1. How does the modular monolith differ from the n-tiered layered architecture?
2. Describe the difference between the peer-to-peer and mediator approaches when communicating between modules.
3. What are the three common risks associated with the modular monolith architectural style?
4. Name three primary strengths of the modular monolith and when you should consider using it.
5. Given its modularity, why aren't operational characteristics such as scalability and fault tolerance good in the modular monolith?

Chapter 12: Pipeline Architecture Style

1. Can pipes be bidirectional in a pipeline architecture?
2. Name the four types of filters and their purpose.
3. Can a filter send data out through multiple pipes?
4. Explain why the pipeline architecture is well suited for cloud-based environments.
5. Is the pipeline architecture style technically partitioned or domain partitioned?
6. In what way does the pipeline architecture support modularity?

Chapter 13: Microkernel Architecture Style

1. What is another name for the microkernel architecture style?

2. Under what situations is it OK for plug-in components to be dependent on other plug-in components?
3. What are some of the tools and frameworks that can be used to manage plug-ins?
4. What would you do if you had a third-party plug-in that didn't conform to the standard plug-in contract in the core system?
5. Provide two examples of the microkernel architecture style.
6. What characteristics of the core system determine its degree of "microkern-ality"?
7. Why is the microkernel architecture always a single architecture quantum?
8. What is *domain/architecture isomorphism*?

Chapter 14: Service-Based Architecture Style

1. Why are services in service-based architecture called *domain services*?
2. Name two common risks associated with service-based architecture.
3. Which database topologies (monolithic, domain, and dedicated) can you use with service-based architecture?
4. What techniques can you use to manage database changes within a service-based architecture?
5. Do domain services require a container (such as Docker) to run?
6. Which architecture characteristics does the service-based architecture style support well?
7. Why isn't elasticity typically well supported in a service-based architecture?
8. How can you increase the number of architecture quanta in a service-based architecture?

Chapter 15: Event-Driven Architecture Style

1. Name four differences between events and messages.
2. What's the difference between an initiating event and a derived event?
3. What is a *poison event*?
4. Can an event processor trigger multiple derived events? If so, why would you want to do this?
5. Why would you trigger an event from an event processor that no other event processor responds to?
6. What are some of the negative trade-offs in using asynchronous processing?
7. Describe the Swarm of Gnats antipattern and explain why you should avoid it.
8. Give two primary reasons why event-driven architecture is highly responsive and has great performance characteristics.

9. What are some of the techniques for preventing data loss when sending and receiving messages from a queue?

Chapter 16: Space-Based Architecture Style

1. Where does space-based architecture get its name from?
2. What is a primary aspect of space-based architecture that differentiates it from other architecture styles?
3. Name the four components that make up the virtualized middleware within a space-based architecture.
4. What is the role of a data writer in space-based architecture?
5. Under what conditions would a service need to access data in a database through the data reader?
6. Does a small cache size increase or decrease the chances for a data collision?
7. What is the difference between a replicated cache and a distributed cache? Which one is typically used in space-based architecture?
8. List the three most strongly supported architecture characteristics in space-based architecture.
9. Why does testability rate so low for space-based architecture?

Chapter 17: Orchestration-Driven Service-Oriented Architecture

1. What was the main driving force behind service-oriented architecture?
2. What are the four primary service types within a service-oriented architecture?
3. List some of the factors that led to the downfall of service-oriented architecture.
4. Is SOA technically partitioned or domain partitioned?
5. How is domain reuse addressed in SOA? How is operational reuse addressed?
6. What is the primary use for this architecture style in modern systems?

Chapter 18: Microservices Architecture

1. Describe the *bounded context* concept in microservices. Why is it so critical for microservices architectures?
2. Why is it so important to isolate data in a microservices architecture?

3. Describe what is meant by *protocol-aware heterogeneous interoperability* and how microservices supports it.
4. What is the difference between orchestration and choreography in microservices? Provide examples where each of these communication styles would be an appropriate choice.
5. Why does microservices use the database-per-service topology? Can it use a monolithic or domain database topology?
6. What are two of the biggest risks in the microservices architecture?
7. Why are agility, testability, and deployability so well supported in microservices?
8. What are three reasons that performance is usually an issue in microservices?
9. Describe a topology where a microservices ecosystem might have only a single quantum.

Chapter 19: Choosing the Appropriate Architecture Style

1. In what way does a system's data architecture (the structure of the logical and physical data models) influence your choice of architecture style?
2. Delineate the steps an architect uses to determine a system's architecture style, data partitioning, and communication styles.
3. What factor leads an architect toward a distributed architecture?
4. What two input analyses does an architecture need to perform to choose an appropriate architectural style?

Chapter 20: Architectural Patterns

1. Name two patterns that implement the architectural concern of separation of operational and domain concerns.
2. What component does an orchestrated workflow have that is absent from choreographed ones?
3. What are some advantages of using a single broker in an event-driven architecture? What are some disadvantages?
4. What two data operations does CQRS break apart?

Chapter 21: Architectural Decisions

1. What is the Covering your Assets antipattern?
2. What are some techniques for avoiding the Email-Driven Architecture antipattern?
3. What are the five factors Michael Nygard defines for identifying something as architecturally significant?
4. What are the five basic sections of an architecture decision record?
5. In which section of an ADR do you typically add the justification for an architecture decision?
6. Assuming you don't need a separate Alternatives section, in which section of an ADR would you list the alternatives to your proposed solution?
7. What are three basic scenarios in which you would mark the status of an ADR as *Proposed*?

Chapter 22: Analyzing Architecture Risk

1. What are the two dimensions of the risk assessment matrix?
2. Describe the three main activities in risk storming.
3. Why does risk storming need to be a collaborative exercise?
4. Why does the identification activity within risk storming need to be an individual activity and not a collaborative one?
5. What would you do if three participants identified risk as high (6) for a particular area of the architecture, but another participant identified it as only medium (3)?
6. What risk rating (1–9) would you assign to unproven or unknown technologies?

Chapter 23: Diagramming Architecture

1. What is *irrational artifact attachment*, and why is it significant with respect to documenting and diagramming architecture?
2. What are the *4 Cs* of the C4 modeling technique?
3. When diagramming architecture, what do dotted lines between components mean?
4. Why is it always important to include a title and key in an architecture diagram?

Chapter 24: Making Teams Effective

1. What are the three types of architect personalities? What type of boundary does each personality create?
2. What five factors should you consider in determining your level of involvement with the team?
3. What are three warning signs that your team is getting too big?
4. List three basic checklists that would be good for a development team to use.

Chapter 25: Negotiation and Leadership Skills

1. Why is negotiation such an important skill for an architect?
2. Name some negotiation techniques for a scenario when a business stakeholder insists on five nines of availability, but only three nines are really needed.
3. What can you derive from a business stakeholder telling you, "I needed it yesterday"?
4. Why is the *demonstration defeats discussion* technique so effective?
5. What is the divide-and-conquer rule? How can it be applied when negotiating architecture characteristics with a business stakeholder? Provide an example.
6. List the 4 Cs of architecture.
7. Explain why it is important for an architect to be both pragmatic and visionary.
8. What are some techniques for managing and reducing the number of meetings you are invited to?

Chapter 26: Architectural Intersections

1. What techniques can you use to ensure structural alignment with the architecture during implementation?
2. Explain why aligning architecture and infrastructure is so important, and provide an example of this alignment.
3. Give an example of when a team's topology might not be aligned with the architecture.
4. Why should an architect in charge of a particular system be concerned about the intersection of architecture and systems integration? Provide an example of where architecture might get misaligned with system integration.
5. What is *domain/architecture isomorphism*, and why is it so important when considering the intersection of architecture and the business?
6. Provide an example of the intersection of architecture and the enterprise.

Chapter 27: The Laws of Software Architecture, Revisited

1. List three generic advantages of a shared library over a shared service. Do these advantages mean an architect should *always* choose a shared library when this trade-off appears?
2. What is the first corollary of the First Law of Software Architecture?
3. Why does the second corollary to the First Law of Software Architecture insist that trade-off analysis must occur over and over?
4. Why are so many software architecture decisions best understood on a spectrum rather than as convenient binaries?

Previous chapter
< 27. The Laws of Software Architecture, Revisited

Next chapter
Index >